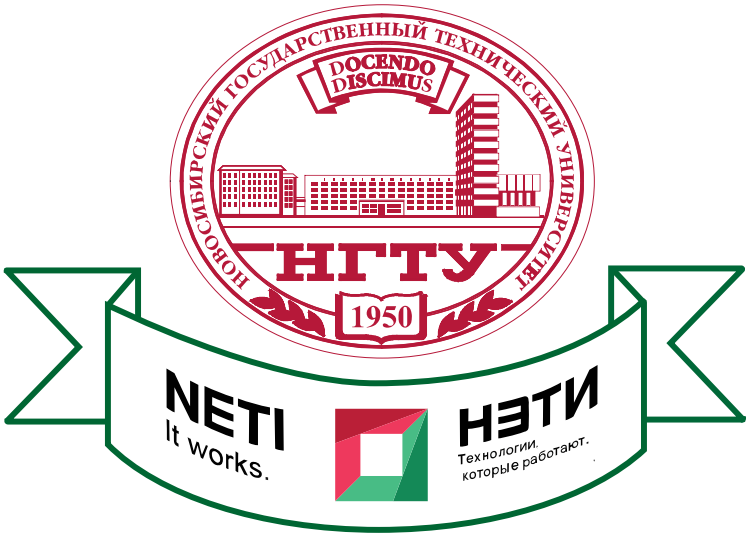
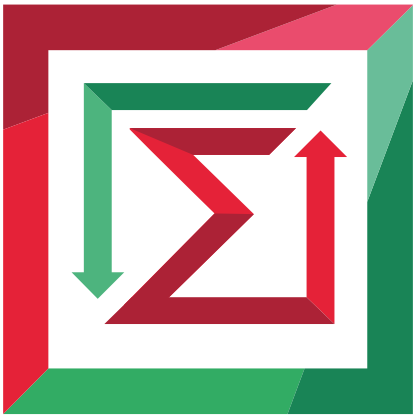
Министерство науки и высшего образования
Российской Федерации
Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



Кафедра теоретической и прикладной информатики
Лабораторная работа №4
По дисциплине «Программные средства защиты информации»

Режимы работы блочных шифров. Схемы кратного шифрования



Факультет:	ПМИ
Группа:	ПМИМ-51
Студенты:	Гавриленко Андрей, Яшков Иван
Вариант:	1
Преподаватель:	Авдеенко Татьяна Владимировна

Новосибирск, 2026

1. Вариант

Алгоритм шифрования из предыдущей работы: AES-128.

№ варианта	Режим шифрования	Схема кратного шифрования
1	CTR	Схема с тремя ключами

2. Задание

I. Реализовать приложение для шифрования, позволяющее выполнять следующие действия:

1. Шифровать данные с использованием заданного в варианте режима шифрования, применённого для того симметричного алгоритма, который был реализован в предыдущей лабораторной работе:

1) шифруемый текст должен храниться в одном файле, ключ шифрования – в другом, а вектор инициализации – в третьем;

2) зашифрованный текст должен сохраняться в файл;

3) в процессе шифрования предусмотреть возможность просмотра и изменения ключа, вектора инициализации, шифруемого и зашифрованного текстов в шестнадцатеричном и символьном виде.

2. Шифровать данные по заданной в варианте схеме кратного шифрования.

3. Исследовать лавинный эффект:

1) приложение может самостоятельно строить необходимые графики либо графики можно строить в стороннем ПО, но тогда приложение для шифрования должно сохранять в файл необходимую для построения графиков информацию.

II. Реализовать приложение для дешифрования, позволяющее выполнять следующие действия:

1. Дешифровать данные с использованием заданного в варианте режима шифрования, применённого для того симметричного алгоритма, который был реализован в предыдущей лабораторной работе:

1) зашифрованный текст должен храниться в одном файле, ключ – в другом, а вектор инициализации – в третьем;

2) расшифрованный текст должен сохраняться в файл;

3) в процессе дешифрования предусмотреть возможность просмотра и изменения ключа, вектора инициализации, зашифрованного и расшифрованного текстов в шестнадцатеричном и символьном виде.

2. Дешифровать данные по заданной в варианте схеме кратного шифрования.

III. С помощью реализованных приложений выполнить следующие задания:

1. Протестировать правильность работы разработанных приложений.

2. Исследовать лавинный эффект для реализованного режима шифрования (рассматривать текст из трёх блоков):

1) построить графики зависимости числа изменённых бит в блоках C_1, C_2, C_3 от позиции изменившегося бита в открытом тексте (3 отдельных графика или 3 зависимости на 1 графике);

2) построить графики зависимости числа изменённых бит в блоках C_1, C_2, C_3 от позиции изменившегося бита в ключе (3 отдельных графика или 3 зависимости на 1 графике);

3) построить графики зависимости числа изменённых бит в блоках C_1, C_2, C_3 от позиции изменившегося бита в векторе инициализации (3 отдельных графика или 3 зависимости на 1 графике);

4) построить графики зависимости числа изменённых бит в блоках P_1, P_2, P_3 от позиции изменившегося бита в зашифрованном тексте (3 отдельных графика или 3 зависимости на 1 графике).
3. Исследовать лавинный эффект для реализованной схемы кратного шифрования (рассматривать текст из 1 блока).
4. Сделать выводы о проделанной работе.

2. Разработанное приложение

При выполнении лабораторной работы было написано консольное приложение на языке C, реализующее все функциональные требования, приведенные в задании под пунктами I, II, III. Некоторые функции программы приведены в конце отчета.
При запуске создаются 7 файлов — message.txt для сообщения, key.txt для ключа при шифровании CTR, init.txt — для IV, key1.txt, key2.txt, key3.txt — ключи для схемы с тремя ключами, cipher.txt для шифротекста.
Основа приложения – блочный шифр AES из 10 раундов с размером блока 128 бит и размером ключа 128 бит, поддерживающий два режима шифрования/дешифрования – CTR и схему с тремя ключами. Схема шифрования с тремя ключами (сокращенно EDE) использует AES-128 с режимом ECB и выглядит так: $C = Ek_3(Dk_2(Ek_1(M)))$.

Главное меню:
Choose a mode:
(1) Encrypt;
(2) Decrypt;
(3) Examine;
(0) Exit.
>

1) При нажатии на (1), пользователь попадает в меню шифрования:
Available files: message.txt/cipher.txt/key.txt/init.txt/key1.txt/key2.txt/key3.txt
Choose a mode:
(1) Encrypt with AES-128 CTR (uses key.txt + init.txt);
(2) Encrypt with AES-128 EDE (uses key1.txt, key2.txt, key3.txt);
(show <file> <16/symbol>) Show a file;
(write <file> <16/symbol> <text>) Write text to file;
(0) Back.
>

Здесь пользователь может задать сообщение, ключи, IV, сообщение шифра с помощью символьного или 16-ичного ввода, посмотреть эти значения в символьном или 16-ичном виде. При нажатии на 1 в файл cipher.txt помещается зашифрованное сообщение в режиме CTR. При нажатии на 2 в файл cipher.txt помещается зашифрованное сообщение в режиме шифрования EDE.

2) При нажатии на (2), выполняется дешифрование текста в cipher.txt:
Choose a mode:
(1) Decrypt with AES-128 CTR;
(2) Decrypt with AES-128 EDE;
(0) Back.
>

При нажатии на 1 в файл message.txt помещается расшифрованное сообщение по алгоритму AES-128 CTR, при нажатии на 2 – по алгоритму AES-128 EDE.

3) При нажатии на 3, пользователь попадает в меню исследования лавинного эффекта:

Choose a mode:

(1) CTR: Flip in message text;
(2) CTR: Flip in key;
(3) CTR: Flip in IV;
(4) CTR: Flip in cipher text;
(avalanche <bit_pos[0..127]>) EDE: flip in P/k1/k2/k3;
(0) Back.

>

При нажатии (1) будет создан CSV файл с данными по заданию III.2.1;
При нажатии (2) будет создан CSV файл с данными по заданию III.2.2;
При нажатии (3) будет создан CSV файл с данными по заданию III.2.3;
При нажатии (4) будет создан CSV файл с данными по заданию III.2.4;
При нажатии (avalanche <номер бита>) будет создано 4 файла с данными по заданию III.3.

3. Задания из пункта 3:

1. Протестировать правильность работы разработанных приложений.

Для тестирования правильности работы алгоритма были взяты входные данные из документа NIST Special Publication 800-38A 2001 Edition, в котором описываются стандарты различных режимов шифрования AES, в том числе и CTR. Там предлагается взять:

Сообщение:	$6bc1bee22e409f96e93d7e117393172a_{16}$
Ключ:	$2b7e151628aed2a6abf7158809cf4f3c_{16}$
IV:	$f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff_{16}$
Должен получаться результат:	$874d6191b620e3261bef6864990db6ce_{16}$

В программе в режиме шифрования были введены команды:

```
> 1                       // Переход в режим шифрования
> write message.txt 16 6bc1bee22e409f96e93d7e117393172a
> write key.txt 16 2b7e151628aed2a6abf7158809cf4f3c
> write init.txt 16 f0f1f2f3f4f5f6f7f8f9fafbfcfdfeff
> 1                       // CTR encrypted 16 bytes.
> show cipher.txt 16      // 874d6191b620e3261bef6864990db6ce
```

Дешифрование показало изначальное сообщение — CTR работает.

Для тестирования EDE никакой документ не использовался — просто было проверено, что дешифрование зашифрованного текста одними и теми же ключами дает изначальное сообщение. Пример использования:

```
> 1                       // Переход в режим шифрования
> write message.txt 16
874d6191b620e3261bef6864990db6ce9806f66b7970fdff8617187bb9fffdff5ae4df3edbd5d35e5b4f09
020db03eab1e031dda2fbe03d1792170a0f3009cee
> write key1.txt 16 ec8cdf7398607cb0f2d21675ea9ea1e4
> write key2.txt 16 362b7c3c6773516318a077d7fc5073ae
> write key3.txt 16 6a2cc3787889374fbeb4c81b17ba6c44
> 2                       // EDE encrypted 64 bytes.
> show cipher.txt 16      //
d64975307e15a9169e24619a5315dd75259281db7a5218510cc958855ea29d4298a67f11fb4278d4cb5879
e88bfd9da4980af808128207ccc40cc9a27041e6da
```

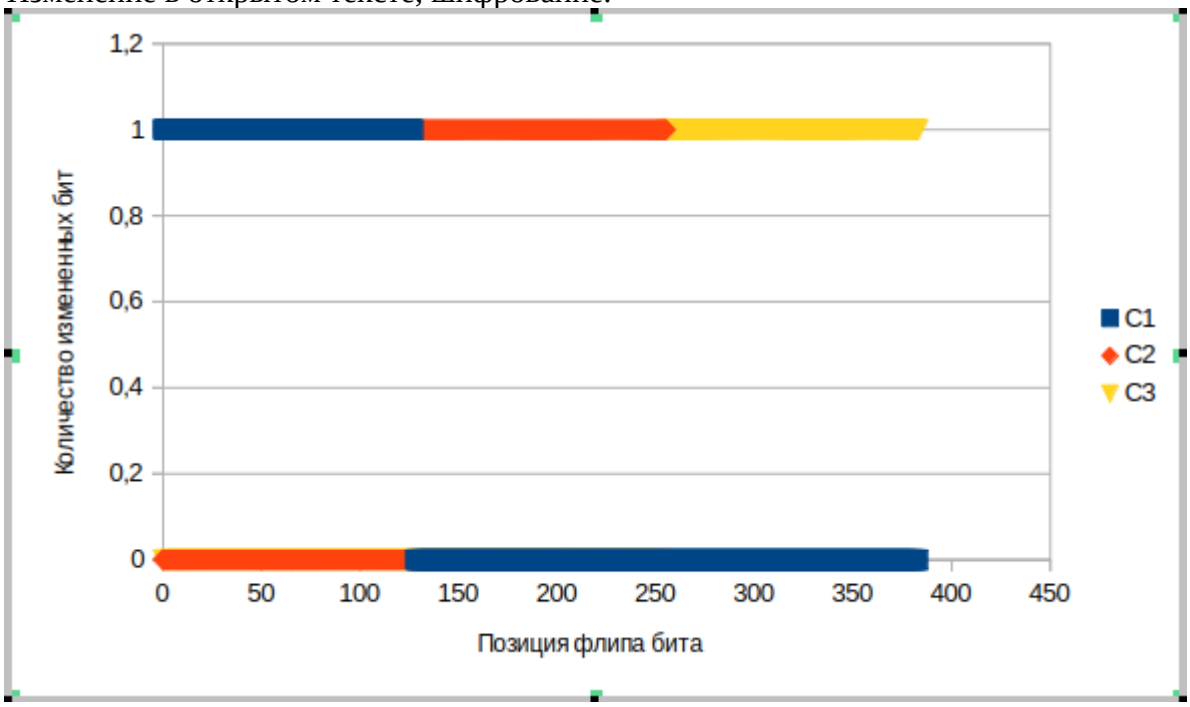
```
> 0                         // Выход из режима шифрования
> 2                         // Переход в режим дешифрования
> 2                         // EDE decrypted 64 bytes
> 0
> 1
> show message.txt 16       //
874d6191b620e3261bef6864990db6ce9806f66b7970fdff8617187bb9fffdff5ae4df3edbd5d35e5b4f09
020db03eab1e031dda2fbe03d1792170a0f3009cee
```

Исходное сообщение совпало.

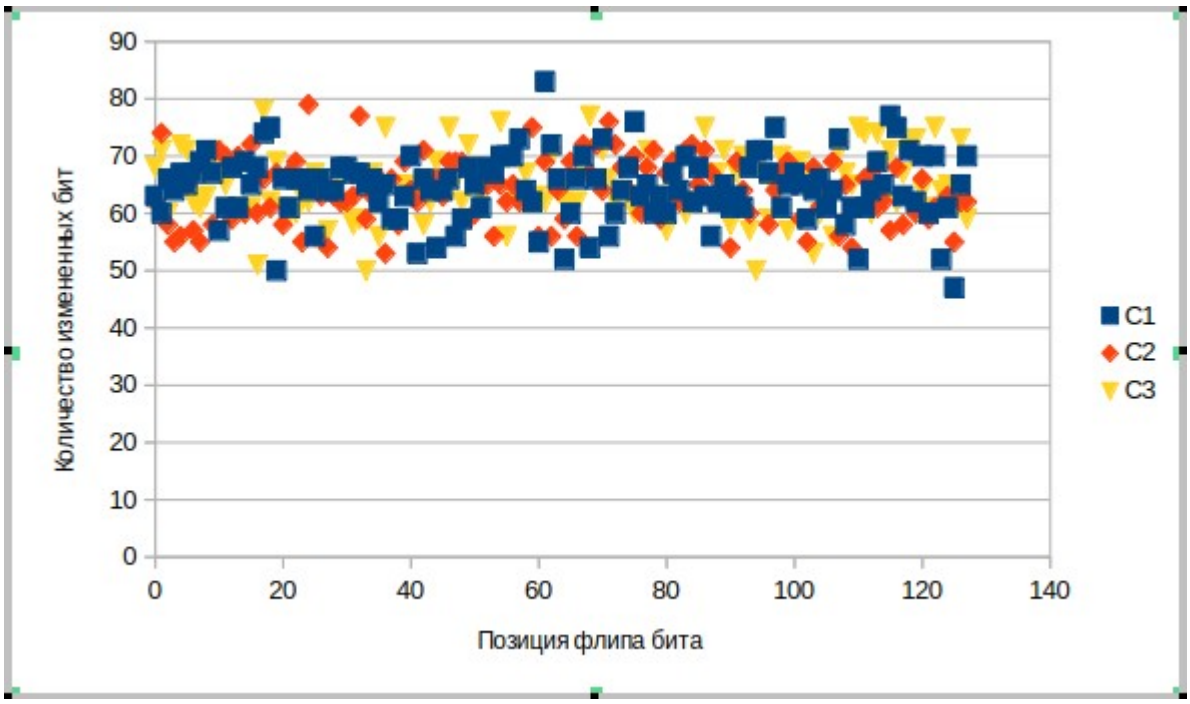
2. Исследовать лавинный эффект для реализованного режима шифрования (рассматривать текст из трёх блоков):
1) построить графики зависимости числа изменённых бит в блоках C_1, C_2, C_3 от позиции изменившегося бита в открытом тексте (3 отдельных графика или 3 зависимости на 1 графике);
2) построить графики зависимости числа изменённых бит в блоках C_1, C_2, C_3 от позиции изменившегося бита в ключе (3 отдельных графика или 3 зависимости на 1 графике);
3) построить графики зависимости числа изменённых бит в блоках C_1, C_2, C_3 от позиции изменившегося бита в векторе инициализации (3 отдельных графика или 3 зависимости на 1 графике);
4) построить графики зависимости числа изменённых бит в блоках P_1, P_2, P_3 от позиции изменившегося бита в зашифрованном тексте (3 отдельных графика или 3 зависимости на 1 графике).
Для тестирования использовался текст из предыдущего пункта:
874d6191b620e3261bef6864990db6ce9806f66b7970fdff8617187bb9fffdff5ae4df3edbd5d35e5b
4f09020db03eab1e031dda2fbe03d1792170a0f3009cee$_{16}$
Текст состоит из 4-х блоков, однако использоваться будут только 3. Для выполнения исследования по каждому пункту сначала выполнялось шифрование (дешифрование для пункта 4) изначального текста с запоминанием значений битов в каждом из трех блоков. Далее, в соотвтетствии с пунктом, выполнялся поочередный флип всех битов в определенном тексте, проводилось повторное шифрование (дешифрование для пункта 4) всех полученных начальных состояний. Результатом каждого пункта является один CSV файл, где каждая строка соответствует конкретному флипу бита, а три столбца отражают, сколько бит в каждом блоке изменилось из-за флипа одного бита. Так как ключ и IV состоят из 128 бит, в 2-3 пунктах количество строк равно 128, в то время как в 1, 4 пунктах количество строк равно 128*3 блока=384.

Результаты:

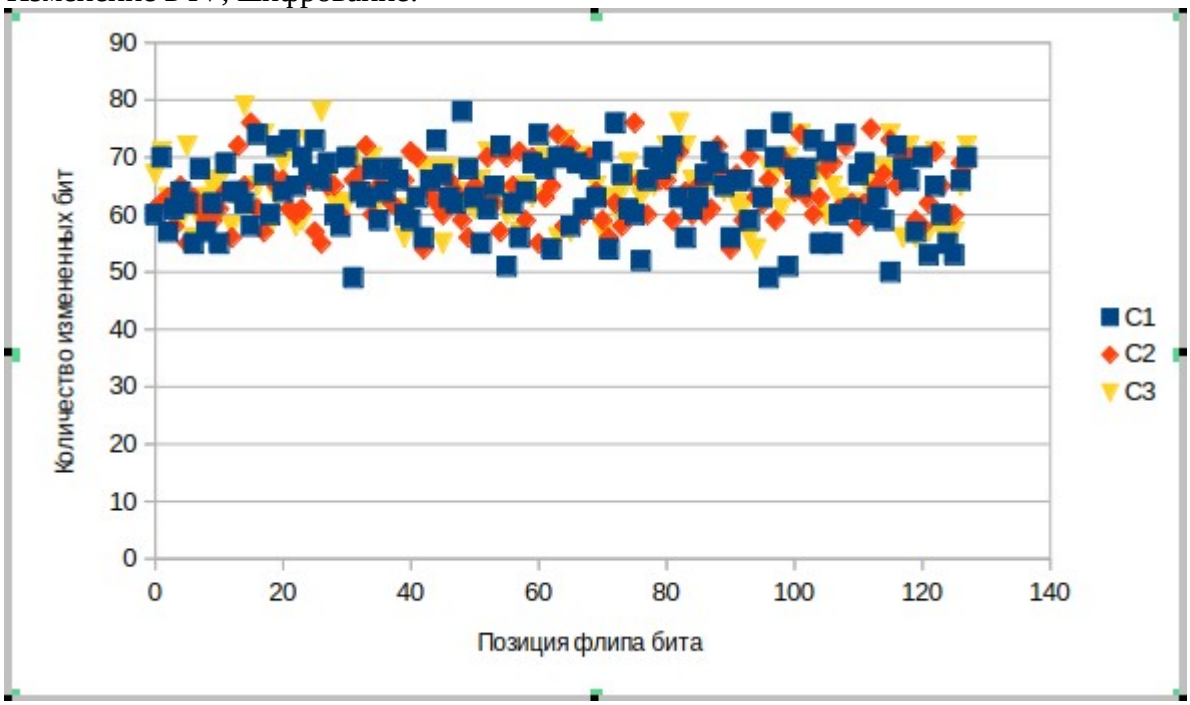
1) Изменение в открытом тексте, шифрование:



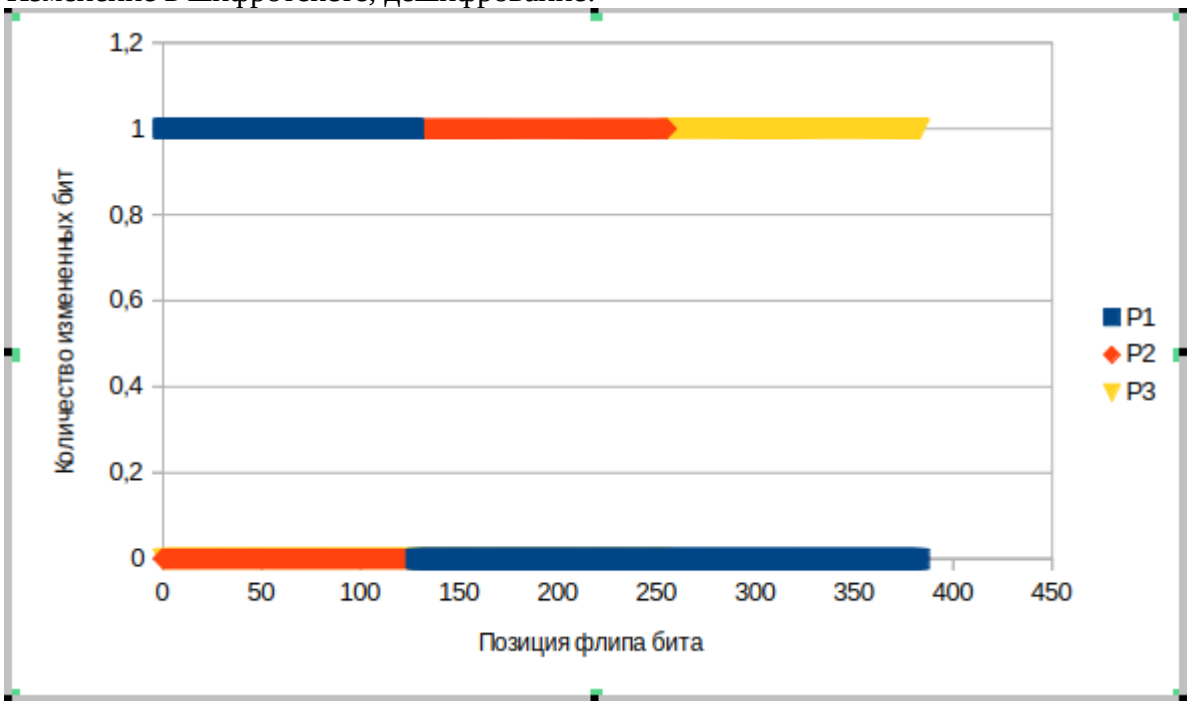
2) Изменение в ключе, шифрование:



3) Изменение в IV, шифрование:



4) Изменение в шифротексте, дешифрование:



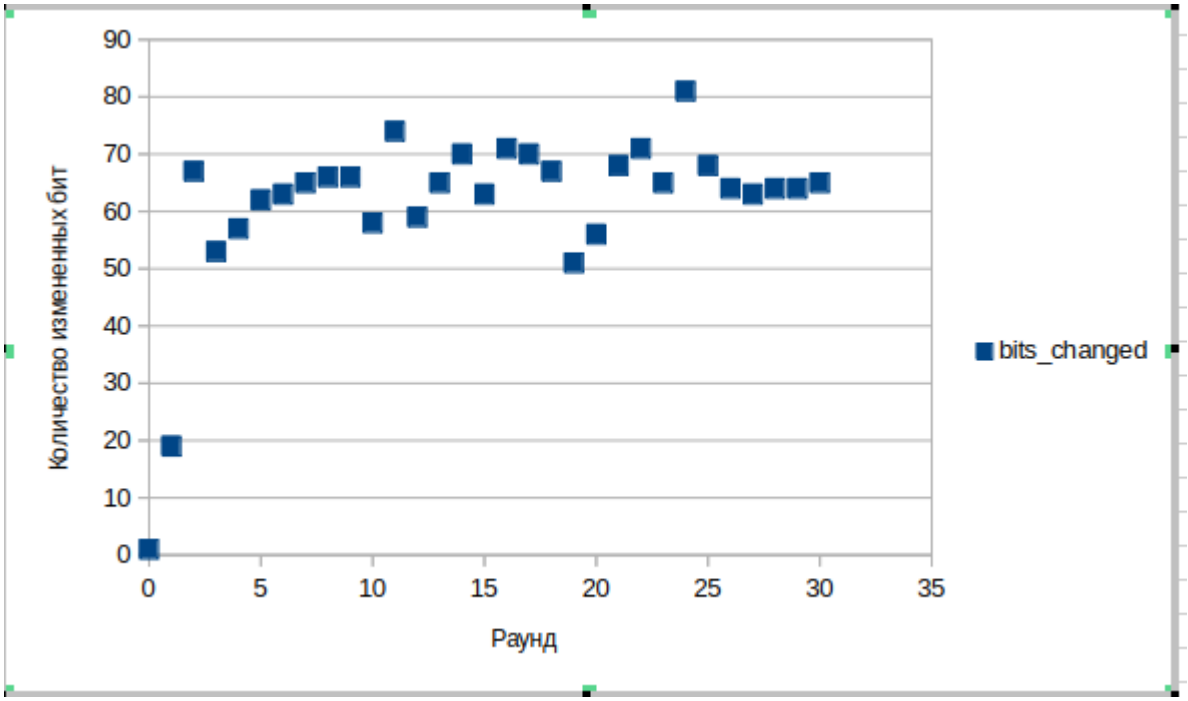
Как можно заметить, в 1 и 4 пунктах при изменении одного бита текста меняется только один бит результата, причем в том же блоке, в котором был произведен флип. Более того, изменяется именно тот бит, над которым был произведен флип — по сути, с режимом CTR блочный шифр становится потоковым. В 2-3 пунктах флип одного бита влияет примерно на полблока данных, что хорошо.

3. Исследовать лавинный эффект для реализованной схемы кратного шифрования (рассматривать текст из 1 блока).

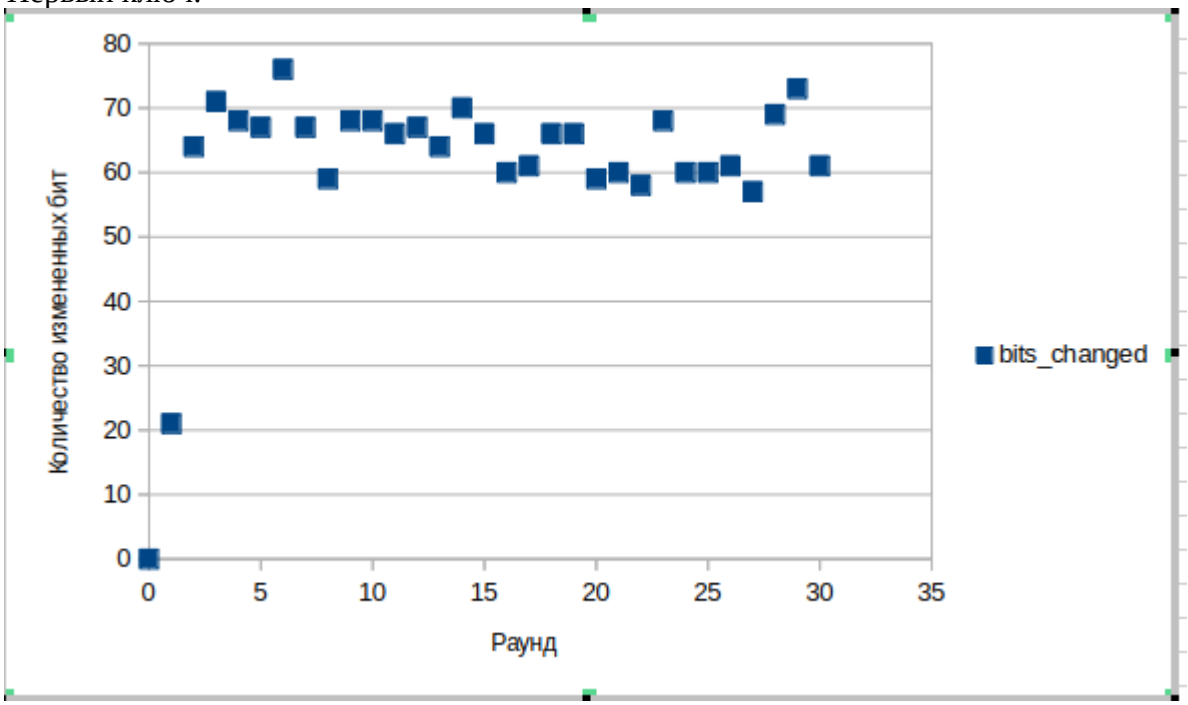
Здесь процедура исследования немного поменялась:

1) Берется текст и три ключа, производится шифрование, запоминается состояние во всех раундах;
2) Производится флип на выбранной позиции к каком-то тексте;
3) Производится повторное шифрование, количество измененных бит смотрится на каждом раунде и сохраняется в файл.
Флип производится отдельно в начальном тексте и во всех ключах.

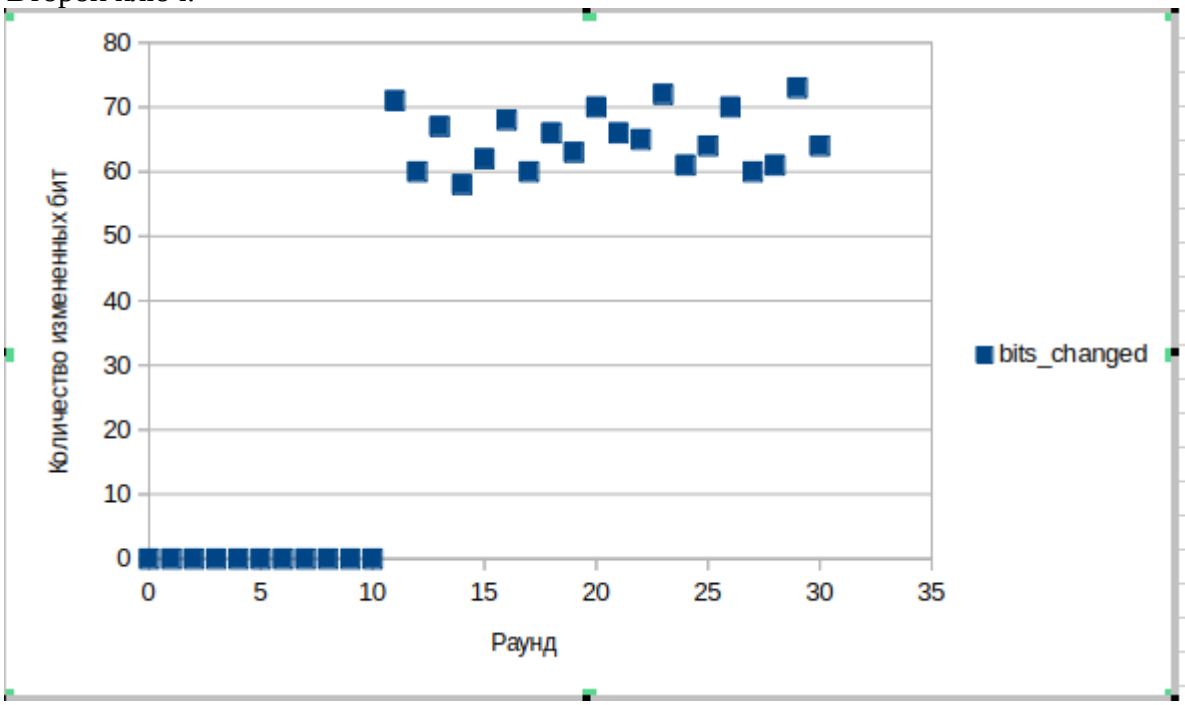
Результат (флип 68 бита):
1) Начальный текст:



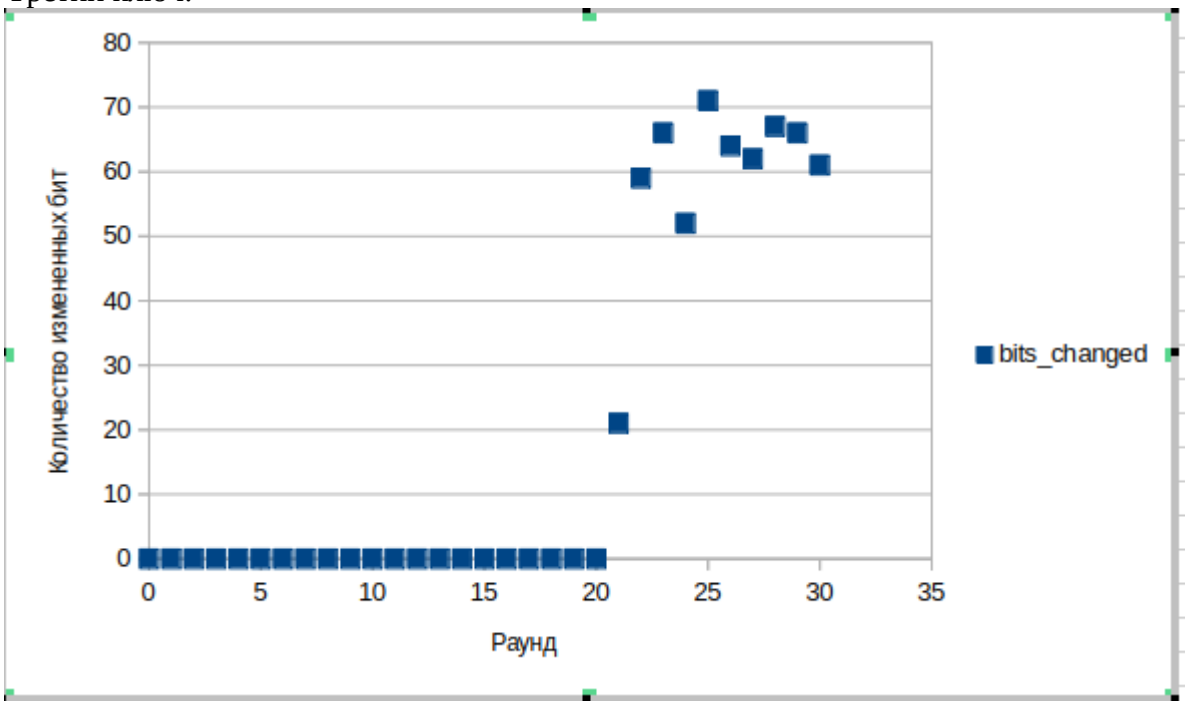
2) Первый ключ:



3) Второй ключ:



4) Третий ключ:



Так как режим EDE построен на трех AES-128 в режиме ECB, то флип одного бита в блоке текста ведет за собой изменение примерно половины всего блока.
Так как второй ключ начинает работать только после 10 раундов с первым ключом, то первые 10 раундов флип бита второго ключа никак не влияет на результат; аналогично с третьим ключом, который начинает влиять только после 20-го раунда.

4. Сделать выводы о проделанной работе.

В данной работе было разработано приложение для шифрования/дешифрования сообщения с помощью AES-128 в режиме CTR и EDE. Глядя на графики исследования лавинного эффекта можно сделать следующие выводы:

1) При использовании режима шифрования CTR, блочный шифр фактически становится потоковым, из-за чего флип одного бита текста/шифротекста не влияет на соседние биты при шифровании/дешифровании;
2) Тем не менее, при изменении одного бита ключа или IV в режиме CTR изменяется примерно полблока данных, что хорошо.
3) Так как режим EDE в данной работе был построен на трех AES-128 в режиме ECB, то флип одного бита в блоке текста ведет за собой изменение примерно половины всего блока – шифр все также остается блочным;
4) В схеме из трех ключей, каждый ключ работает только свои 10 раундов: первый – первые 10, второй – вторые, третий – третие. Потому флип бита третьего ключа будет по факту влиять только на последние 10 раундов, а не на все 30.

4. Несколько основных функций

1) Функция шифрования/дешифрования в режиме CTR:

```
void ctr_crypt_buffer(const uint8_t *in, size_t in_len, uint8_t *out,
                      const uint8_t key[AES_KEY_BYTES],
                      const uint8_t iv[AES_BLOCK_BYTES]) {
  uint8_t counter[AES_BLOCK_BYTES];
  memcpy(counter, iv, AES_BLOCK_BYTES);

  uint8_t gamma[AES_BLOCK_BYTES];
  size_t full = in_len / AES_BLOCK_BYTES;
  size_t tail = in_len % AES_BLOCK_BYTES;

  for (size_t b = 0; b < full; b++) {
    aes_encrypt_block(counter, gamma, key);
    for (int i = 0; i < AES_BLOCK_BYTES; i++)
      out[b * AES_BLOCK_BYTES + i] = in[b * AES_BLOCK_BYTES + i] ^ gamma[i];
    ctr_increment(counter);
  }

  if (tail > 0) {
    aes_encrypt_block(counter, gamma, key);
    for (size_t i = 0; i < tail; i++)
      out[full * AES_BLOCK_BYTES + i] =
          in[full * AES_BLOCK_BYTES + i] ^ gamma[i];
  }
}
```

2) Функция шифрования блока в режиме EDE:

```
void ede_encrypt_block(const uint8_t in[AES_BLOCK_BYTES],
                       uint8_t out[AES_BLOCK_BYTES],
                       const uint8_t k1[AES_KEY_BYTES],
                       const uint8_t k2[AES_KEY_BYTES],
                       const uint8_t k3[AES_KEY_BYTES]) {
  uint8_t t1[16], t2[16];
  aes_encrypt_block(in, t1, k1);
  aes_decrypt_block(t1, t2, k2);
  aes_encrypt_block(t2, out, k3);
}
```

3) Функция дешифрования блока EDE:

```
void ede_decrypt_block(const uint8_t in[AES_BLOCK_BYTES],
                       uint8_t out[AES_BLOCK_BYTES],
                       const uint8_t k1[AES_KEY_BYTES],
                       const uint8_t k2[AES_KEY_BYTES],
                       const uint8_t k3[AES_KEY_BYTES]) {
  uint8_t t1[16], t2[16];
  aes_decrypt_block(in, t1, k3);
  aes_encrypt_block(t1, t2, k2);
  aes_decrypt_block(t2, out, k1);
}
```

4) Функция исследования лавинного эффекта для CTR:

```
static int run_ctr_avalanche(int target) {
  uint8_t msg[AVAL_BYTES] = {0};
  uint8_t key[16] = {0};
  uint8_t iv[16] = {0};

  load_msg_3blocks(msg);
  if (read_key16(KEY_FILE, key) == FAILURE)
    return FAILURE;
  if (read_key16(INIT_FILE, iv) == FAILURE)
    return FAILURE;

  uint8_t g_base[AVAL_BLOCKS][16];
  ctr_gamma3(key, iv, g_base[0], g_base[1], g_base[2]);

  uint8_t C_base[AVAL_BYTES];
  for (int b = 0; b < AVAL_BLOCKS; b++)
    for (int i = 0; i < 16; i++)
      C_base[b * 16 + i] = msg[b * 16 + i] ^ g_base[b][i];

  uint8_t P_base[AVAL_BYTES];
  for (int b = 0; b < AVAL_BLOCKS; b++)
    for (int i = 0; i < 16; i++)
      P_base[b * 16 + i] = C_base[b * 16 + i] ^ g_base[b][i];

  int total_bits = (target == 2 || target == 3) ? 128 : 384;

  char fname[64];
  const char *tag;
  if (target == 1) {
    tag = "ctr_avalanche_1.csv";
    snprintf(fname, sizeof(fname), "%s", tag);
  } else if (target == 2) {
    tag = "ctr_avalanche_2.csv";
    snprintf(fname, sizeof(fname), "%s", tag);
  } else if (target == 3) {
    tag = "ctr_avalanche_3.csv";
    snprintf(fname, sizeof(fname), "%s", tag);
  } else {
    tag = "ctr_avalanche_4.csv";
    snprintf(fname, sizeof(fname), "%s", tag);
  }

  int fd = open(fname, O_WRONLY | O_CREAT | O_TRUNC, 0644);
  if (fd == -1) {
    return FAILURE;
  }

  dprintf(fd, target == 4 ? "bit_pos,P1,P2,P3\n" : "bit_pos,C1,C2,C3\n");

  for (int bit = 0; bit < total_bits; bit++) {
    uint8_t msg_m[AVAL_BYTES];
    uint8_t key_m[16], iv_m[16];
    uint8_t C_m[AVAL_BYTES];
    uint8_t gamma_m[AVAL_BLOCKS][16];

    memcpy(msg_m, msg, AVAL_BYTES);
    memcpy(key_m, key, 16);
    memcpy(iv_m, iv, 16);
    memcpy(gamma_m, g_base, sizeof(gamma_m));

    if (target == 1) {
      flip_bit(msg_m, bit);
      for (int b = 0; b < AVAL_BLOCKS; b++)
        for (int i = 0; i < 16; i++)
          C_m[b * 16 + i] = msg_m[b * 16 + i] ^ g_base[b][i];
    } else if (target == 2) {
```

```
      flip_bit(key_m, bit);
      ctr_gamma3(key_m, iv, gamma_m[0], gamma_m[1], gamma_m[2]);
      for (int b = 0; b < AVAL_BLOCKS; b++)
        for (int i = 0; i < 16; i++)
          C_m[b * 16 + i] = msg[b * 16 + i] ^ gamma_m[b][i];
    } else if (target == 3) {
      flip_bit(iv_m, bit);
      ctr_gamma3(key, iv_m, gamma_m[0], gamma_m[1], gamma_m[2]);
      for (int b = 0; b < AVAL_BLOCKS; b++)
        for (int i = 0; i < 16; i++)
          C_m[b * 16 + i] = msg[b * 16 + i] ^ gamma_m[b][i];
    } else {
      memcpy(C_m, C_base, AVAL_BYTES);
      flip_bit(C_m, bit);
    }

    int d1, d2, d3;
    if (target == 4) {
      for (int b = 0; b < AVAL_BLOCKS; b++)
        for (int i = 0; i < 16; i++)
          C_m[b * 16 + i] = C_m[b * 16 + i] ^ g_base[b][i];

      d1 = popcount_xor(C_m, P_base, 16);
      d2 = popcount_xor(C_m + 16, P_base + 16, 16);
      d3 = popcount_xor(C_m + 32, P_base + 32, 16);
    } else {
      d1 = popcount_xor(C_m, C_base, 16);
      d2 = popcount_xor(C_m + 16, C_base + 16, 16);
      d3 = popcount_xor(C_m + 32, C_base + 32, 16);
    }

    dprintf(fd, "%d,%d,%d,%d\n", bit, d1, d2, d3);
  }

  close(fd);
  printf("  wrote %s\n", fname);
  return SUCCESS;
}
```

5) Функция исследования лавинного эффекта для EDE:

```
static int run_ede_avalanche(int bit_pos) {
  static uint8_t msg_raw[BUFFER_SIZE];
  size_t m_len = 0;

  if (read_bytes(MESSAGE_FILE, msg_raw, BUFFER_SIZE, &m_len) == FAILURE)
    return FAILURE;

  uint8_t P[16] = {0};
  memcpy(P, msg_raw, m_len < 16 ? m_len : 16);

  uint8_t k1[16], k2[16], k3[16];
  if (read_key16(KEY1_FILE, k1) == FAILURE)
    return FAILURE;
  if (read_key16(KEY2_FILE, k2) == FAILURE)
    return FAILURE;
  if (read_key16(KEY3_FILE, k3) == FAILURE)
    return FAILURE;

  uint8_t base_states[EDE_STATES][16];
  uint8_t base_out[16];
  ede_encrypt_block_track(P, base_out, k1, k2, k3, base_states);

  const char *tags[4] = {"p", "k1", "k2", "k3"};

  for (int t = 0; t < 4; t++) {
    uint8_t P_m[16], k1_m[16], k2_m[16], k3_m[16];
    memcpy(P_m, P, 16);
    memcpy(k1_m, k1, 16);
```

```
    memcpy(k2_m, k2, 16);
    memcpy(k3_m, k3, 16);

    if (t == 0)
      flip_bit(P_m, bit_pos);
    else if (t == 1)
      flip_bit(k1_m, bit_pos);
    else if (t == 2)
      flip_bit(k2_m, bit_pos);
    else
      flip_bit(k3_m, bit_pos);

    uint8_t mod_states[EDE_STATES][16];
    uint8_t mod_out[16];
    ede_encrypt_block_track(P_m, mod_out, k1_m, k2_m, k3_m, mod_states);

    char fname[96];
    snprintf(fname, sizeof(fname), "avalanche_ede_%s_bit%d.csv", tags[t],
             bit_pos);

    int fd = open(fname, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
      return FAILURE;
    }

    dprintf(fd, "round,bits_changed\n");
    for (int r = 0; r < EDE_STATES; r++) {
      int d = popcount_xor(base_states[r], mod_states[r], 16);
      dprintf(fd, "%d,%d\n", r, d);
    }
    close(fd);
    printf("  wrote %s\n", fname);
  }
  return SUCCESS;
}
```